# Worksheet No. 6

**Student Name:** ANINDITA DHAR **UID:** 25MCA20259
**Branch:** MCA (GENERAL) **Section/Group:** MCA- 1-A
**Semester: II** **Date of Performance:** 27/01/2026

**Subject Name:** Technical Training **Subject Code:** 25CAP-652

## 1.Aim/Overview of the practical:

Learn how to create, query, and manage views in SQL to simplify database queries and provide a layer of abstraction for end-users.

## 2. Objectives:

- **Data Abstraction:** To understand how to hide complex table joins and calculations behind a simple virtual table interface.
- **Enhanced Security:** To learn how to restrict user access to sensitive columns by providing views instead of direct table access.
- **Query Simplification:** To master the creation of views that pre-join multiple tables, making reporting easier for non-technical users.
- **View Management:** To understand the syntax for creating, altering, and dropping views, as well as the naming conventions required for efficient data access.

## 3. Input/Apparatus Used:

- PostgreSQL
- pgAdmin

## 4. Practical / Experiment Steps

**Step 1: Creating a Simple View for Data Filtering**
Implementing a view to provide a quick list of active employees without exposing the entire table structure.

**Step 2: Creating a View for Joining Multiple Tables**
Simplifying the retrieval of data distributed across the Employees and Departments tables.

**Step 3: Advanced Summarisation View**
Creating a view to provide department-level statistics automatically

## 5.Procedure/Algorithm/Code:

i.Start the system and log in to the computer.
ii. Open PostgreSQL software.

iii. **Create a table using the DDL command.**

```
CREATE TABLE Departments_ (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR (50)
);

CREATE TABLE Employees_5(
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    salary NUMERIC,
    status VARCHAR(10),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);
```

**iv. Insert records into both tables.**

```
INSERT INTO Departments_ VALUES
(1, 'HR')
INSERT INTO Departments_ VALUES
(2, 'IT')
INSERT INTO Departments_ VALUES
(3, 'Finance')

INSERT INTO  Employees_5 VALUES
(1, 'Roshan', 30000, 'Active', 2)
INSERT INTO  Employees_5 VALUES
(2, 'Anindita', 40000, 'Active', 2)
INSERT INTO  Employees_5 VALUES
(3, 'Shriya', 25000, 'Inactive', 1)
INSERT INTO  Employees_5 VALUES
(4, 'Anju', 35000, 'Active', 3)
INSERT INTO  Employees_5 VALUES
(5, 'Sanjana', 28000, 'Active', 1);
```

**v.**

```
CREATE VIEW active_employees AS
SELECT emp_id, emp_name, dept_id
FROM Employees_5
WHERE status = 'Active';

SELECT * FROM active_employees;
```

| | emp_id integer | emp_name character varying (50) | dept_id integer |
|---|---|---|---|
| 1 | 1 | Roshan | 2 |
| 2 | 2 | Anindita | 2 |
| 3 | 4 | Anju | 3 |
| 4 | 5 | Sanjana | 1 |

**viii.**

```
CREATE VIEW employee_department_view AS
SELECT e.emp_id, e.emp_name, d.dept_name
FROM employee e
JOIN department d ON e.dept_id = d.dept_id;

SELECT * FROM employee_department_view;
```

| | emp_id integer | emp_name character varying (20) | dept_name character varying (20) |
|---|---|---|---|
| 1 | 101 | Amit Sharma | Engineering |
| 2 | 102 | Neha Verma | Engineering |
| 3 | 104 | Priya Mehta | Marketing |
| 4 | 103 | Rohit Singh | Finance |

**ix.**

```
CREATE VIEW department_summary AS
SELECT d.dept_name,
      COUNT(e.emp_id) AS total_employees,
      AVG(e.salary) AS average_salary
FROM department d
JOIN employee e ON d.dept_id = e.dept_id
GROUP BY d.dept_name;
```

SELECT * FROM department_summary;

| | dept_name character varying (50) | total_employees bigint | average_salary numeric |
|---|---|---|---|
| 1 | Finance | 1 | 35000.000000000000 |
| 2 | IT | 2 | 35000.000000000000 |
| 3 | HR | 1 | 28000.000000000000 |

DROP VIEW department_summary;

```
DROP VIEW

Query returned successfully in 97 msec.
```

## 7. Learning Outcomes

- **Abstraction Proficiency:** Students will be able to create and query views to simplify efficient data access and abstraction.
- **Security Implementation:** Students will understand how to use views for data masking and providing restricted access to sensitive information.
- **Syntactic Accuracy:** Students will demonstrate the correct syntax for creating and querying views, ensuring logical clarity in naming conventions.
- **Real-world Application:** Students will be able to design views for practical domains like Library Management Systems or Payroll Systems to demonstrate functionality.